

TUGAS MANDIRI MODUL 8

DASAR KECERDASAN ARTIFISIAL



**Universitas
Telkom**

Dipersiapkan Oleh :
Rovino Ramadhani – 103072400031

UNIVERSITAS TELKOM SURABAYA
PROGRAM STUDI INFORMATIKA
2026

Code

<https://github.com/rovinor4/Praktikum-Dasar-Kecerdasan-Artifisial/tree/8259ab502094326eb6600abae21e23338b8979ef/Tugas%20Mandiri%20-%20Modul%208%20Algoritma%20Hill%20Climbing>

Penjelasan Dan Output Soal 1

```
In [30]: import math
import random

import matplotlib.pyplot as plt
```

Kode ini mengimpor library yang dibutuhkan. math digunakan untuk menghitung jarak Euclidean, random digunakan untuk mengacak rute awal, dan matplotlib.pyplot digunakan untuk menampilkan visualisasi rute dalam bentuk grafik.

```
In [31]: cities = [
    (0, 0),
    (2, 3),
    (5, 1),
    (6, 4),
    (8, 0),
    (1, 7),
    (4, 6)
]
```

Kode ini mendefinisikan daftar kota dalam bentuk koordinat (x, y). Setiap elemen pada list cities mewakili posisi satu kota yang akan dikunjungi dalam proses pencarian rute.

```
In [32]: def total_distance(route):
    distance = 0

    for i in range(len(route)):
        city_now = cities[route[i]]
        city_next = cities[route[(i + 1) % len(route)]]
        distance += euclidean_distance(city_now, city_next)

    return distance
```

Fungsi total_distance(route) digunakan untuk menghitung total jarak dari sebuah rute. Fungsi ini menjumlahkan jarak antar kota secara berurutan, termasuk jarak dari kota terakhir kembali ke kota pertama.

```
In [33]: def get_best_neighbor(route):
    best_neighbor = route[:]
    best_distance = total_distance(route)

    for i in range(len(route)):
        for j in range(i + 1, len(route)):
            neighbor = route[:]
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]

            neighbor_distance = total_distance(neighbor)

            if neighbor_distance < best_distance:
                best_neighbor = neighbor
```

```
best_distance = neighbor_distance
```

```
return best_neighbor, best_distance
```

Fungsi `get_best_neighbor(route)` digunakan untuk mencari rute tetangga terbaik. Caranya adalah dengan menukar posisi dua kota dalam rute, lalu memilih rute dengan total jarak paling kecil.

```
In [34]: def hill_climbing():
    current_route = list(range(len(cities)))
    random.shuffle(current_route)

    initial_route = current_route[:]
    initial_distance = total_distance(initial_route)

    while True:
        neighbor, neighbor_distance = get_best_neighbor(current_route)
        current_distance = total_distance(current_route)

        if neighbor_distance < current_distance:
            current_route = neighbor
        else:
            print("Solusi optimal lokal telah ditemukan")
            break

    return initial_route, initial_distance, current_route, total_distance(current_route)
```

Fungsi `hill_climbing()` menjalankan algoritma Hill Climbing. Rute awal dibuat secara acak, lalu program terus mencari tetangga yang lebih baik sampai tidak ditemukan rute dengan jarak yang lebih pendek. Jika sudah tidak ada perbaikan, program berhenti pada solusi optimal lokal.

```
In [35]: def route_to_coordinates(route):
    x = [cities[i][0] for i in route]
    y = [cities[i][1] for i in route]

    x.append(cities[route[0]][0])
    y.append(cities[route[0]][1])

    return x, y
```

Fungsi `route_to_coordinates(route)` mengubah urutan rute menjadi daftar koordinat x dan y. Kota pertama ditambahkan kembali di akhir agar grafik membentuk rute tertutup.

```
In [36]: def euclidean_distance(city1, city2):
    return math.sqrt((city1[0] - city2[0])**2 + (city1[1] - city2[1])**2)
```

Fungsi `euclidean_distance(city1, city2)` menghitung jarak lurus antara dua kota. Rumus yang digunakan adalah rumus jarak Euclidean berdasarkan selisih koordinat x dan y.

```
In [37]: def plot_route(route, title):
    x, y = route_to_coordinates(route)

    plt.figure(figsize=(7, 5))
    plt.plot(x, y, marker="o")

    for i, city_index in enumerate(route):
        plt.text(cities[city_index][0], cities[city_index][1], f"K{city_index}")

    plt.title(title)
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.grid(True)
    plt.show()
```

Fungsi `plot_route(route, title)` digunakan untuk menampilkan visualisasi rute. Setiap kota ditampilkan sebagai titik, diberi label, lalu dihubungkan dengan garis sesuai urutan rutanya.

```
In [38]: initial_route, initial_distance, best_route, best_distance = hill_climbing()
```

Solusi optimal lokal telah ditemukan

Kode ini menjalankan fungsi `hill_climbing()`. Hasilnya disimpan ke dalam variabel rute awal, jarak awal, rute terbaik, dan jarak terbaik setelah proses optimasi.

```
In [39]: print("Rute awal:")
    print(initial_route)

    print("\nRute terbaik hasil optimasi:")
    print(best_route)

    print("\nTotal jarak sebelum optimasi:")
    print(round(initial_distance, 2))

    print("\nTotal jarak sesudah optimasi:")
    print(round(best_distance, 2))

    plot_route(initial_route, "Visualisasi Rute Awal")
    plot_route(best_route, "Visualisasi Rute Terbaik Hasil Hill Climbing")
```

Rute awal:

```
[1, 4, 6, 3, 0, 5, 2]
```

Rute terbaik hasil optimasi:

```
[1, 5, 6, 3, 4, 2, 0]
```

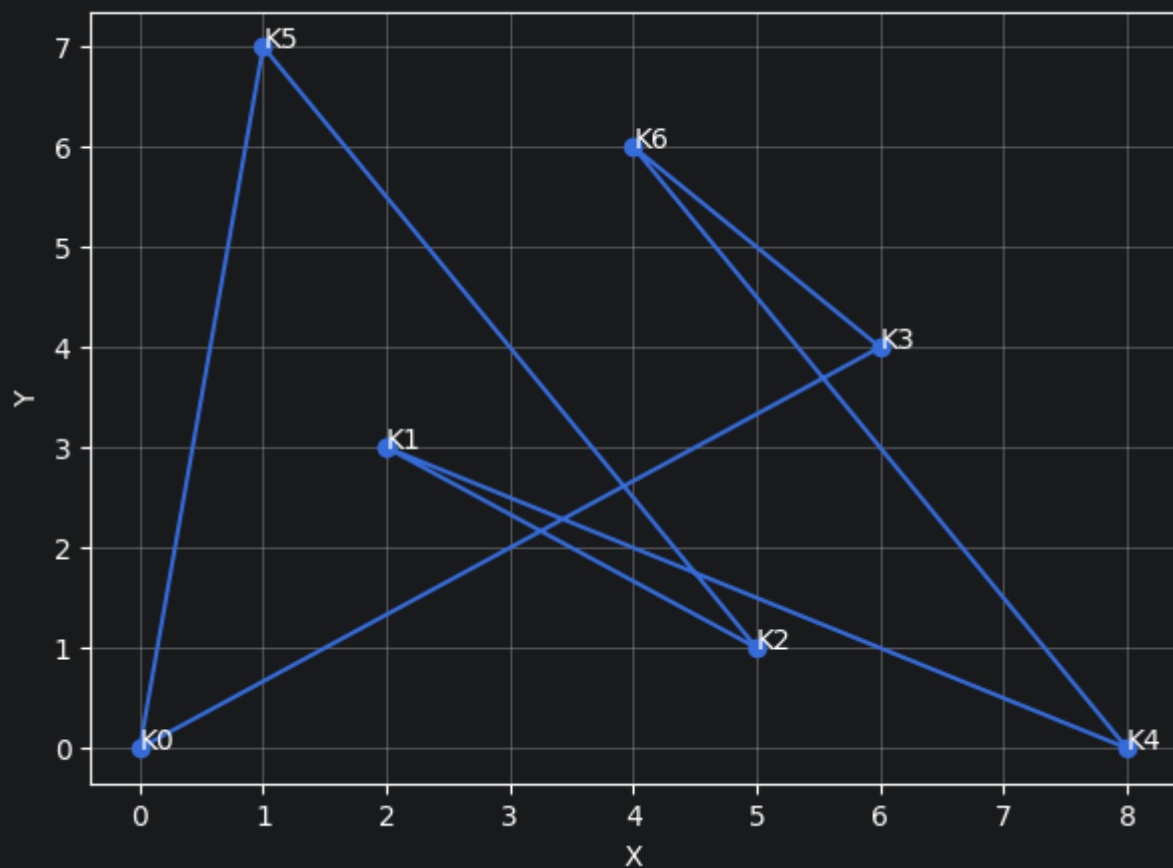
Total jarak sebelum optimasi:

```
41.85
```

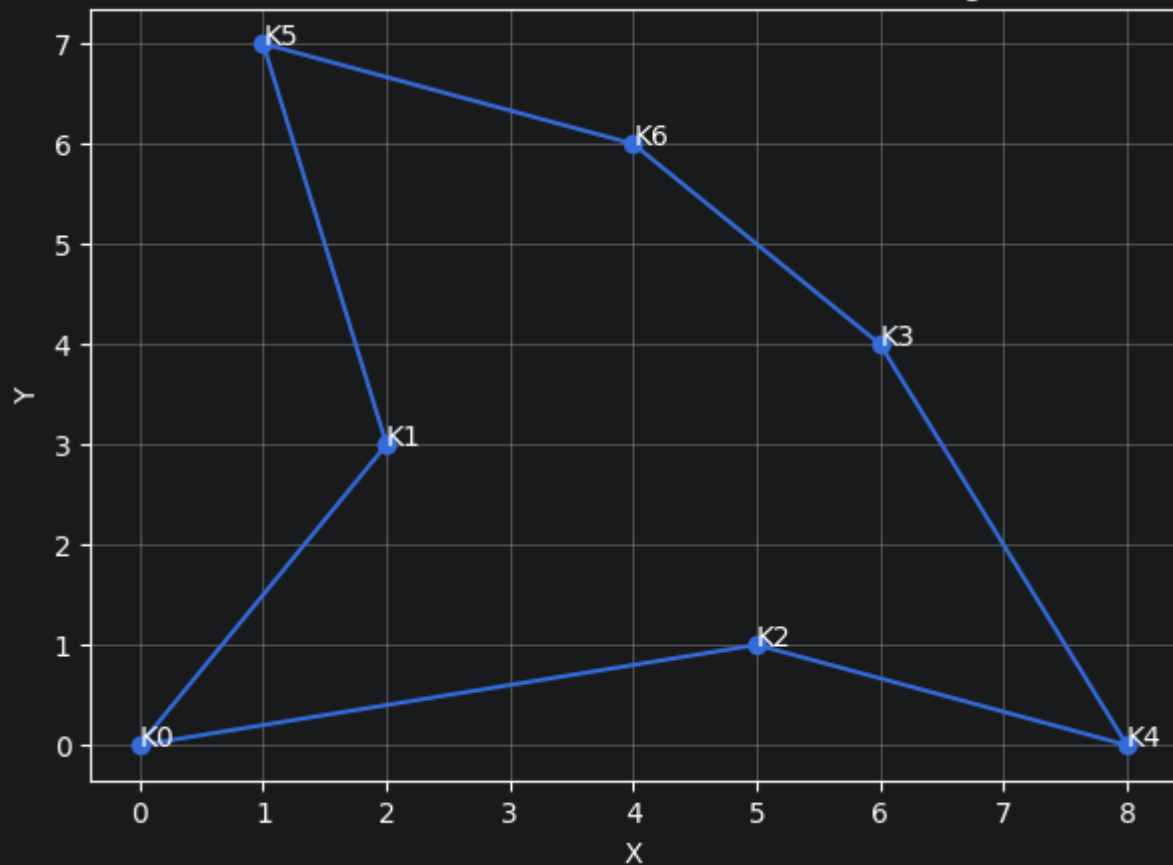
Total jarak sesudah optimasi:

```
26.45
```

Visualisasi Rute Awal



Visualisasi Rute Terbaik Hasil Hill Climbing



Kode ini menampilkan hasil akhir program, yaitu rute awal, rute terbaik, total jarak sebelum optimasi, dan total jarak setelah optimasi. Setelah itu, program menampilkan dua grafik untuk membandingkan rute awal dan rute terbaik hasil Hill Climbing.

Penjelasan Dan Output Soal 2

```
In [1]: import random

import matplotlib.pyplot as plt
```

Kode ini mengimpor library yang dibutuhkan. random digunakan untuk mengacak urutan tugas awal, sedangkan matplotlib.pyplot digunakan untuk membuat grafik perubahan cost setiap iterasi.

```
In [2]: tasks = ['A', 'B', 'C', 'D', 'E']

task_duration = {
    'A': 4,
    'B': 2,
    'C': 7,
    'D': 3,
    'E': 5
}

task_priority = {
    'A': 3,
    'B': 5,
    'C': 2,
    'D': 4,
    'E': 1
}
```

Kode ini mendefinisikan daftar tugas, durasi setiap tugas, dan prioritas setiap tugas. Data ini menjadi dasar untuk menghitung biaya penjadwalan.

```
In [3]: def calculate_cost(schedule):
    total_cost = 0
    completion_time = 0

    for task in schedule:
        completion_time += task_duration[task]
        total_cost += completion_time * task_priority[task]

    return total_cost
```

Fungsi calculate_cost(schedule) digunakan untuk menghitung total cost dari suatu urutan tugas. Cost dihitung berdasarkan waktu selesai setiap tugas yang dikalikan dengan prioritas tugas tersebut.

```
In [4]: def get_best_neighbor(schedule):
    best_neighbor = schedule[:]
    best_cost = calculate_cost(schedule)

    for i in range(len(schedule)):
        for j in range(i + 1, len(schedule)):
            neighbor = schedule[:]
            neighbor[i], neighbor[j] = neighbor[j], neighbor[i]

            neighbor_cost = calculate_cost(neighbor)

            if neighbor_cost < best_cost:
                best_neighbor = neighbor
                best_cost = neighbor_cost
```

```
return best_neighbor, best_cost
```

Fungsi `get_best_neighbor(schedule)` digunakan untuk mencari susunan tugas tetangga terbaik. Program mencoba menukar posisi dua tugas, lalu memilih susunan dengan cost paling kecil.

```
In [5]: def hill_climbing():
    current_schedule = tasks[:]
    random.shuffle(current_schedule)

    initial_schedule = current_schedule[:]
    initial_cost = calculate_cost(initial_schedule)

    iteration = 0
    history = []

    print("Solusi awal:", initial_schedule)
    print("Cost awal:", initial_cost)
    print()

    history.append(initial_cost)

    while True:
        current_cost = calculate_cost(current_schedule)
        neighbor, neighbor_cost = get_best_neighbor(current_schedule)

        if neighbor_cost < current_cost:
            iteration += 1
            current_schedule = neighbor
            history.append(neighbor_cost)

            print("Iterasi ke-", iteration)
            print("Solusi:", current_schedule)
            print("Cost:", neighbor_cost)
            print()
        else:
            print("Solusi optimal lokal telah ditemukan")
            break

    return initial_schedule, initial_cost, current_schedule, calculate_cost(current_s
```

Fungsi `hill_climbing()` menjalankan algoritma Hill Climbing. Program membuat susunan tugas awal secara acak, lalu terus mencari susunan baru yang memiliki cost lebih kecil sampai tidak ada perbaikan lagi.

```
In [6]: def plot_cost(history):
    plt.figure(figsize=(7, 5))
    plt.plot(range(len(history)), history, marker="o")
    plt.title("Perubahan Cost Setiap Iterasi")
    plt.xlabel("Iterasi")
    plt.ylabel("Cost")
    plt.grid(True)
    plt.show()
```

Fungsi `plot_cost(history)` digunakan untuk menampilkan grafik perubahan cost dari setiap iterasi. Grafik ini membantu melihat apakah nilai cost semakin menurun selama proses optimasi.

```
In [7]: initial_schedule, initial_cost, best_schedule, final_cost, history = hill_climbing()

print("Solusi terbaik:", best_schedule)
print("Total cost akhir:", final_cost)
```

Solusi awal: ['D', 'E', 'C', 'B', 'A']
Cost awal: 198

Iterasi ke- 1
Solusi: ['D', 'B', 'C', 'E', 'A']
Cost: 141

Iterasi ke- 2
Solusi: ['D', 'B', 'A', 'E', 'C']
Cost: 120

Iterasi ke- 3
Solusi: ['B', 'D', 'A', 'E', 'C']
Cost: 113

Iterasi ke- 4
Solusi: ['B', 'D', 'A', 'C', 'E']
Cost: 110

Solusi optimal lokal telah ditemukan
Solusi terbaik: ['B', 'D', 'A', 'C', 'E']
Total cost akhir: 110

Kode ini menjalankan fungsi `hill_climbing()`, lalu menampilkan solusi terbaik dan total cost akhir.
Hasil proses juga disimpan ke dalam variabel `history` untuk divisualisasikan.

```
In [8]: plot_cost(history)
```



Kode ini memanggil fungsi `plot_cost(history)` untuk menampilkan grafik perubahan cost. Grafik menunjukkan penurunan cost dari solusi awal sampai solusi optimal lokal ditemukan.